

Aula 3 — Interfaceamento paralelo e display

O controlador HD44780 e o custo da espera

Microcontroladores — DENE/UFMT

OBJETIVOS

- Reconhecer um barramento paralelo como um contrato de temporização entre linhas de dado, de endereço e de habilitação — e distingui-lo do acionamento direto de pino do encontro 2.
- Descrever a organização do controlador HD44780: memória de exibição, gerador de caracteres e a separação entre registrador de instrução e registrador de dados.
- Implementar a comunicação em quatro bits justificando cada atraso da sequência de inicialização a partir da folha de dados, e não por cópia.
- Estimar em ciclos de máquina o tempo consumido por uma atualização de tela e reconhecer o display como o periférico mais lento do sistema.
- Aplicar estratégias de atualização que evitam tremulação e reduzem o bloqueio do processador.

1 Do pino ao barramento

Acionar um pino foi, até aqui, a coisa mais simples do curso. Escrever 1 em LATD0 acende o LED, e ele fica aceso. O **nível** é a informação: quem observar o pino em qualquer instante lê o mesmo estado, e não existe momento privilegiado.

O display quebra isso, e essa é a mudança conceitual do encontro.

Escrever um valor em LATD não transfere nada para o display. O valor fica parado nos fios, e o controlador o ignora — até que uma segunda linha, a de habilitação, produza uma borda. É a borda que transfere. O nível apenas prepara.

Um barramento paralelo é um **contrato de temporização**. Ele reparte os fios em três papéis:

CONCEITO

dado — as linhas que carregam o valor (aqui, D7–D4);

endereço — a linha que diz o que o valor significa (RS: um bit de endereço, que escolhe entre dois registradores do controlador);

habilitação — a linha que declara o instante em que o dado é válido (E).

O contrato diz quanto tempo antes da borda o dado precisa estar estável, quanto tempo depois ele precisa permanecer, e qual é a largura mínima do pulso. Violar qualquer um desses números não produz erro: produz um caractere errado, às vezes.

O par de registradores selecionado por RS é a mesma ideia dos SFRs do encontro 1 — um registrador alcançado por um endereço —, realizada por outro mecanismo. Lá o endereço vinha do campo f da instrução; aqui vem de um fio. A ideia sobrevive à mudança de implementação, e é por isso que ela merecia um nome.

NOTA

1.1 A conta de pinos

Com as três linhas de controle acionáveis, o display consome:

Modo	Pinos	Composição
Oito bits	11	D0–D7, mais RS, E e R/W
Quatro bits	7	D4–D7, mais RS, E e R/W
Quatro bits, R/W no terra	6	D4–D7, mais RS e E

A terceira linha da tabela é o que a maioria das placas faz, e o que este kit não fez. A seção 2 mostra o que se compra com o pino a mais.

Quatro pinos de diferença decidem o projeto do semestre. Com 34 pinos de E/S e a necessidade de acomodar sensor, aquecedor, ventoinha, teclado matricial e comunicação, o modo reduzido não é uma otimização: é o que torna o termostato viável no kit.

NOTA

Note também o que a conta **não** diz. É comum ouvir que o paralelo é mais rápido que o serial; não é, e o mundo dos periféricos modernos é a prova.

A distinção que importa é outra: **quem gera o protocolo**. Aqui é o software — cada borda de E é uma instrução sua, e o processador fica preso enquanto isso. No encontro 10 o mesmo trabalho passa a um periférico dedicado, que produz as bordas sozinho enquanto o programa faz outra coisa. Essa é a fronteira real, e ela não tem nada a ver com o número de fios.

O próprio PIC18F4550 tem um barramento paralelo **no outro papel**: a PORTD pode operar como porta paralela escrava, com linhas de seleção e de habilitação vindas de fora. Nessa configuração o microcontrolador é o periférico, e quem produz as bordas é outro processador.

NOTA

Mesmo contrato, papéis invertidos. Vale saber que existe, porque explica por que a PORTD tem funções alternativas com nomes estranhos na tabela de pinos.

2 O controlador HD44780

O display do laboratório não é uma matriz de pontos comandada diretamente: é um módulo com controlador próprio, que recebe comandos e caracteres e cuida da varredura, da geração dos desenhos e da atualização.

Elemento	Função
Memória de exibição (DDRAM)	Guarda os códigos dos caracteres mostrados. Cada posição da tela corresponde a um endereço fixo
Gerador de caracteres em ROM	Contém os desenhos dos caracteres padrão, indexados pelo código recebido
Gerador de caracteres em RAM	Permite definir caracteres próprios — tipicamente o símbolo de grau, ausente do conjunto padrão
Registrador de instrução (IR)	Recebe comandos: limpar, posicionar cursor, configurar modo
Registrador de dados (DR)	Recebe os códigos dos caracteres a exibir

E três linhas de controle governam a comunicação:

Linha	Nome usual	Significado
RS	Seleção de registrador	Nível baixo escolhe o IR (comando); nível alto escolhe o DR (dado)
R/W	Leitura ou escrita	Nível baixo escreve; nível alto lê. Aterrará-lo economiza um pino e desiste da leitura
E	Habilitação	O dado é capturado na borda de descida . É o pulso que efetiva a transferência

A montagem da placa decide o resto da aula, e esta placa não faz o que a maioria faz. As três linhas de controle estão na PORTE:

NESTE KIT

Linha	Pino	
RS	RE0	Seleção de registrador
E	RE1	Habilitação
R/W	RE2	Acionável por software — a leitura está disponível

A via de dados ocupa a PORTD inteira; em quatro bits usam-se RD4–RD7.

Isto foi medido, não lido. O manual do kit e o código de exemplo que circula com ele já divergiram quanto a este mapeamento, e a folha de dados do módulo não pode ajudar:

ela descreve o controlador, não onde alguém o soldou. O procedimento de verificação está na página do curso — e é o mesmo tipo de medida que vocês repetem no R4.

RE0–RE2 são também AN5–AN7. Se saem do reset em modo analógico ou digital depende dos bits PCFG do ADCON1, cujo valor de reinicialização é escolhido por um **bit de configuração** — que neste kit pertence ao bootloader, não ao seu programa.

ATENÇÃO

Neste kit o display escreve sem que ninguém toque no ADCON1. Não conclua daí que o pino é digital por natureza. Conclua que o código que veio antes do seu já resolveu isso — e que no chip nu da protoboard do encontro 0, onde a configuração é sua, o mesmo programa pode não escrever nada.

Três qualidades de espera. A pergunta “a operação anterior já terminou?” aparece três vezes no curso, com três respostas de qualidade crescente:

CONCEITO

esperar às cegas — o programa conta um tempo fixo, maior que o pior caso da folha de dados, e não pergunta nada. É o que resta quando R/W está no terra;

perguntar — o programa lê o indicador de ocupado e espera ele baixar. Custa um pino e algum código, e devolve o tempo **real** em vez do pior caso. É o que este kit permite, e o que a seção 4.3 implementa;

ser avisado — o dispositivo sinaliza a conclusão sem ser consultado. É o ACK do I²C no encontro 11.

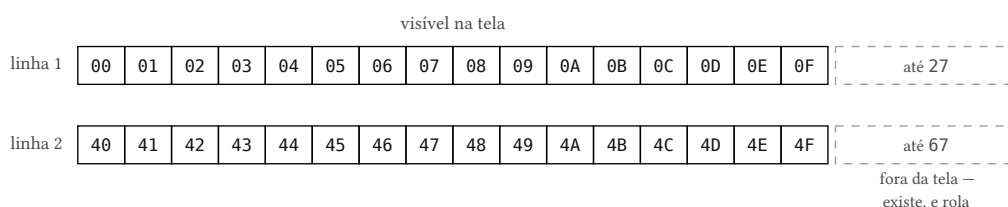
Entre a primeira e a segunda a diferença é de informação. Entre a segunda e a terceira, de quem gasta o tempo esperando: perguntar ainda ocupa o processador com a pergunta.

Quase todo módulo que você encontrar fora deste laboratório tem R/W aterrado, e quase todo driver publicado espera às cegas por causa disso. O driver desta aula funciona nos dois casos, porque a leitura é opcional — mas um driver copiado da internet não vai usar o pino que esta placa pagou para ter.

DIVERGÊNCIA

2.1 Endereçar a tela

Os endereços da memória de exibição não são contíguos entre as linhas.



A segunda linha não começa em 10: começa em 40. O endereço é da memória do controlador, não da posição na tela.

Figura 1: A memória do controlador é maior que a tela: são 40 posições por linha, das quais 16 aparecem. As demais existem, guardam o que for escrito nelas, e entram em cena com os comandos de rolagem.

A descontinuidade é a origem de um defeito clássico: escrever dezessete caracteres seguidos **não** faz o texto passar para a segunda linha — ele continua em posições invisíveis da primeira. Mudar de linha exige um comando explícito de posicionamento.

ATENÇÃO

3 O contrato, em números

A folha de dados do HD44780 especifica um ciclo de escrita com cinco tempos mínimos. Todos eles valem para o modo de quatro bits sem alteração: o que muda é que cada byte gasta dois ciclos em vez de um.

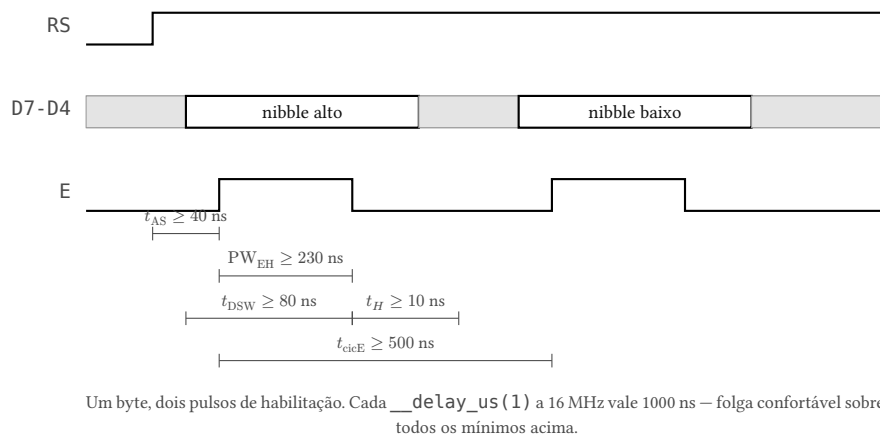


Figura 2: O ciclo de escrita de um byte em quatro bits. Os números são os mínimos da folha de dados a 5 V.

De onde sai o `__delay_us(1)`. A 16 MHz, um ciclo de máquina vale 250 ns, e `__delay_us(1)` consome quatro deles: 1000 ns. Compare com os mínimos:

CONCEITO

$PW_{EH} \geq 230 \text{ ns}$, $t_{AS} \geq 40 \text{ ns}$, $t_{DSW} \geq 80 \text{ ns}$, $t_H \geq 10 \text{ ns}$, $t_{cicE} \geq 500 \text{ ns}$.

O menor múltiplo de T_{cy} que satisfaz todos é **um único ciclo de máquina**, 250 ns — e ele já bastaria para quatro dos cinco. O `__delay_us(1)` é escolhido por outro motivo: dá folga de mais de quatro vezes sobre o pior caso, o que absorve variação de módulo, de temperatura e de tensão sem que ninguém precise medir nada.

Isto é diferente de copiar o valor de um exemplo da internet. O número foi escolhido **sabendo** qual restrição ele atende e com que margem.

Os 230 ns valem para operação a 5 V. Módulos alimentados a 2,7 V, e boa parte dos clones vendidos hoje, especificam $PW_{EH} \geq 450 \text{ ns}$. Ainda cabe no `__delay_us(1)`, mas não caberia num pulso de um ciclo de máquina.

DIVERGÊNCIA

Este é o argumento contra “otimizar” o driver reduzindo o pulso: o ganho é de 750 ns por nibble, e o preço é um driver que funciona na sua bancada e falha na do colega, com um módulo de outro lote.

4 O modo de quatro bits

Cada byte vai em duas etapas: primeiro os quatro bits mais significativos, depois os quatro menos significativos, cada metade com seu próprio pulso de habilitação.

```
static void lcd_pulso(void)
{
    LATEbits.LATE1 = 1;      /* E em nível alto */
    __delay_us(1);           /* PW_EH: mínimo 230 ns */
    LATEbits.LATE1 = 0;      /* borda de descida: captura */
    __delay_us(1);           /* t_cicE: mínimo 500 ns entre subidas */
}

static void lcd_nibble(uint8_t valor)
{
    LATD = (uint8_t)((LATD & 0x0F) | (valor & 0xF0));
    lcd_pulso();
}

void lcd_escrever(uint8_t valor, uint8_t eh_dado)
{

```

```

LATEbits.LATE0 = eh_dado;          /* RS */
LATEbits.LATE2 = 0;                /* R/W: escrita */
lcd_nibble(valor);                 /* parte alta */
lcd_nibble((uint8_t)(valor << 4)); /* parte baixa */
__delay_us(40);                    /* execucao: uma vez por byte */
}

```

A espera de execução vale por **byte**, não por nibble. O controlador só começa a executar quando o segundo nibble chega; esperar 40 µs depois do primeiro é esperar por nada.

ATENÇÃO

O erro é fácil de cometer — basta pôr o atraso dentro de `lcd_pulso`, onde ele parece pertencer — e dobra o custo de toda a interface. Uma tela inteira passa de 1,3 ms para 2,7 ms sem que nada pare de funcionar.

4.1 Por que LATD e não PORTD

A primeira linha de `lcd_nibble` lê LATD para preservar a parte baixa enquanto altera a parte alta. Trocar por PORTD reintroduz um defeito que o curso ainda não nomeou.

Leitura-modificação-escrita. Toda operação sobre um bit de porta — LATD — é 0x10, BSF, LATDbits.LATD4 = 1 — é na verdade três operações: ler os oito bits, alterar um, escrever os oito de volta.

CONCEITO

Se a leitura vem de PORTD, o que se lê é o **estado elétrico dos pinos**, não o que foi escrito neles. Um pino carregado por capacitância, por um LED, ou pela entrada de outro chip pode não ter chegado ao nível escrito no instante da leitura — e o valor lido, errado, é reescrito nos outros sete pinos.

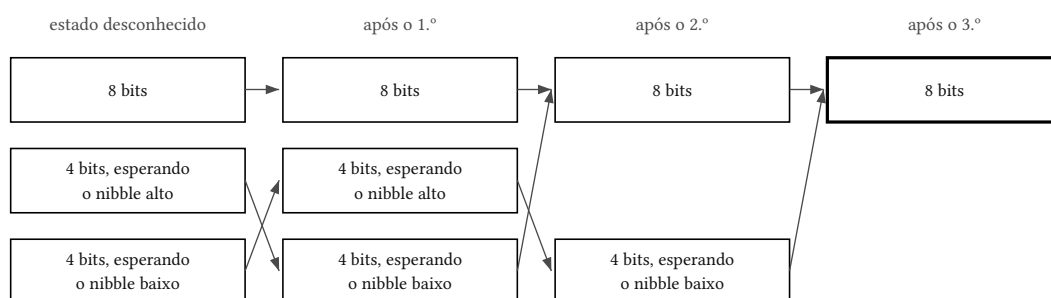
LATD lê o **latch**: o que o programa mandou, independentemente do que o mundo fez com o fio. É por isso que a família PIC18 tem LAT, e a família de faixa média não tinha.

Este defeito é primo do “LAT antes de TRIS” do encontro 2, e tem a mesma assinatura: nada trava, o pino errado muda, e o sintoma depende da carga elétrica ligada — ou seja, some quando você desconecta as coisas para investigar.

NOTA

4.2 A sequência de inicialização

O controlador nasce em modo de oito bits, mas **reiniciar o microcontrolador não reinicia o display**. Depois de um reset por SW9, com o módulo ainda alimentado, o controlador pode estar em qualquer um de três estados.



Cada seta é um pulso de 0x3 em D7–D4. O estado que espera o nibble baixo completa o byte 0x33 — que é justamente “interface de 8 bits”. Três repetições porque três é o número de estados possíveis, e o programa não sabe em qual deles começou.

Figura 3: A inicialização é uma resincronização às cegas. O programa não pode perguntar em que estado o controlador está — só pode enviar uma sequência que leva os três ao mesmo lugar.

São **três** estados possíveis, não dois: oito bits; quatro bits esperando o nibble alto; quatro bits esperando o nibble baixo. O último existe porque um reset pode ocorrer entre os dois pulsos de um mesmo byte.

CONCEITO

Um pulso de 0x3 em D7–D4 age diferente em cada um:

em oito bits, é o comando “interface de 8 bits” — nada muda;

esperando o alto, vira o nibble alto — passa a esperar o baixo;

esperando o baixo, completa um byte com o nibble alto anterior, executa lixo, e volta a esperar o alto.

Só na **segunda** rodada o estado “esperando o baixo” completa o byte 0x33, que é exatamente “interface de 8 bits”. Daí três repetições: três estados, e a convergência custa três passos.

```
void lcd_iniciar(void)
{
    __delay_ms(50);           /* inicializacao interna do modulo */

    lcd_nibble(0x30); __delay_ms(5);
    lcd_nibble(0x30); __delay_us(150);
    lcd_nibble(0x30); __delay_us(150);
    lcd_nibble(0x20);         /* a partir daqui, modo de 4 bits */

    lcd_comando(0x28);        /* 4 bits, 2 linhas, matriz 5x8 */
    lcd_comando(0x0C);        /* display ligado, sem cursor */
    lcd_comando(0x06);        /* avanco automatico do cursor */
    lcd_comando(0x01);        /* limpar */
    __delay_ms(2);           /* limpar demora mais que os demais */
}
```

Os três atrasos da sequência têm origens diferentes, e nenhum é arbitrário.

CONCEITO

50 ms — não são para o display: são para a **alimentação**. O controlador só garante o estado interno depois que VDD estabiliza, e quem liga o kit pela USB não controla quanto tempo isso leva.

5 ms depois do primeiro 0x30 — porque o byte de lixo completado pelo estado “esperando o baixo” pode ser 0x03, que é o comando “retornar ao início” e custa cerca de 1,5 ms. Cinco milissegundos cobrem o pior caso com folga tripla. Este é o único atraso da sequência que existe por causa de um comando que **ninguém quis executar**.

150 µs nas duas repetições seguintes — a partir daí não há mais byte de lixo possível, e basta o tempo de um comando comum, com margem.

4.3 Ler o indicador de ocupado

Com R/W acionável, a espera fixa de 40 µs deixa de ser a única opção. O controlador sabe dizer se ainda está ocupado; basta inverter o sentido do barramento e perguntar.

```
static uint8_t lcd_ocupado(void)
{
    uint8_t alto, baixo;

    TRISD |= 0xF0;           /* D7-D4 viram entrada */
    LATEbits.LATE0 = 0;      /* RS = 0: registrador de instrucao */
    LATEbits.LATE2 = 1;      /* R/W = 1: leitura */
    __delay_us(1);

    LATEbits.LATE1 = 1; __delay_us(1);
    alto = (uint8_t)(PORTD & 0xF0);
    LATEbits.LATE1 = 0; __delay_us(1);
```

```

LATEbits.LATE1 = 1; __delay_us(1);
baixo = (uint8_t)((PORTD & 0xF0) >> 4);
LATEbits.LATE1 = 0; __delay_us(1);

LATEbits.LATE2 = 0;          /* devolve o barramento antes do TRIS */
TRISD &= 0x0F;
return (uint8_t)(alto & 0x80); /* D7 é o indicador de ocupado */
}

static void lcd_esperar(void)
{
    while (lcd_ocupado()) {
        ;                      /* o tempo real, não o pior caso */
    }
}

```

Aqui PORTD está certo e LATD estaria errado. É o único lugar do driver em que isso acontece, e a razão é exatamente a da seção 4.1, lida ao contrário.

CONCEITO

Lá se queria o que o programa escreveu, e o latch responde. Aqui se quer o que **outro circuito** colocou no fio, e o latch não sabe de nada — ele guarda a última escrita do PIC, não a resposta do módulo. Ler LATD devolveria o nibble que o programa mandou da última vez.

Um mesmo par de registradores, duas perguntas opostas, e a resposta certa é diferente em cada uma. Quem decorou “sempre LAT” erra aqui.

A ordem das duas últimas linhas não é decorativa. Devolver o barramento (R/W = 0) antes de reconfigurar o TRISD garante que o módulo nunca esteja dirigindo os mesmos fios que o PIC.

ATENÇÃO

Neste caso específico o módulo já soltou o barramento, porque ele só o dirige enquanto E está alto — e E já desceu. A ordem é disciplina defensiva, não correção de um defeito ativo. Vale mantê-la porque em outros barramentos do curso a folga não existe, e porque a alternativa é dois circuitos dirigindo o mesmo fio: a única falha desta aula que danifica hardware em vez de produzir caractere errado.

lcd_esperar não pode ser usada **durante** a sequência de inicialização. Antes que os três pulsos de 0x30 terminem, o controlador não está num modo conhecido e a leitura de dois nibbles não significa nada. Os atrasos da seção 4.2 continuam sendo cegos por necessidade — é justamente o estado em que perguntar não é possível.

ATENÇÃO

Depois da inicialização, lcd_esperar substitui o __delay_us(40) de lcd_escrever e, principalmente, o __delay_ms(2) da limpeza — onde a diferença entre o pior caso e o tempo real é de milissegundos.

5 O custo em tempo

A maioria dos comandos exige cerca de 37 μs. Escrever dezesseis caracteres numa linha custa

CONCEITO

$$16 \cdot 37 \mu\text{s} \approx 0,6 \text{ ms}$$

Somando o posicionamento de cursor de cada linha e considerando as duas linhas, uma atualização completa de tela custa da ordem de **1,3 ms**.

O comando de limpeza, sozinho, exige cerca de 1,5 ms — quarenta vezes mais que um comando comum, porque ele percorre as 80 posições da memória de exibição.

Em ciclos de máquina, a 250 ns por ciclo:

Operação	Tempo	Ciclos	Comparação
Um comando ou caractere	40 μ s	160	—
Atualização de tela	1,3 ms	5200	O mais caro do sistema
Limpar a tela	1,5 ms	6000	E ainda pisca
Uma conversão do ADC	15 μ s	60	Afirmado aqui; derivado no encontro 4

As duas primeiras linhas desta tabela vêm da folha de dados, e são **piores casos**. Com `lcd_esperar` no lugar do atraso fixo, o kit devolve o tempo do módulo que está na sua bancada — que costuma ser menor, e que varia de módulo para módulo. Medir os dois e comparar é o Exercício 3.8.

NOTA

O display é, com folga, o periférico mais lento do sistema — quase duas ordens de grandeza acima de tudo o mais. Qualquer discussão sobre desempenho neste projeto passa por ele, e por nenhum outro.

ATENÇÃO

A consequência é contraintuitiva o suficiente para valer como regra: **quem quiser mais tempo de processador não deve medir menos — deve escrever menos**.

5.1 Duas consequências de projeto

Não limpar a tela a cada atualização. Limpar e reescrever custa os 6000 ciclos do comando somados aos 5200 da escrita, e produz tremulação visível: por um instante a tela está em branco. A prática correta é reposicionar o cursor e sobrescrever, mantendo o comprimento do texto constante e completando com espaços quando o número encolhe — caso contrário restam dígitos antigos à direita.

Não atualizar mais rápido do que se consegue ler. Atualizar a cada 1 ms é desperdício puro: o olho não acompanha, e o custo é o mais alto do sistema. Uma atualização a cada 200 ms a 500 ms é confortável e reduz o consumo de tempo em duas ordens de grandeza.

```
/* Escrever apenas quando o valor mudou. */
static int16_t exibido = INT16_MIN;

void atualizar_display(int16_t decimos)
{
    if (decimos == exibido) {
        return; /* nada a fazer: sai imediatamente */
    }
    exibido = decimos;

    lcd_posicao(0, 0);
    lcd_texto("T=");
    lcd_numero_fixo(decimos); /* largura constante */
    lcd_texto(" C ");
}
```

Guardar o que já foi mostrado e escrever apenas a diferença é a versão elementar de uma ideia que reaparece em toda interface gráfica, de displays de duas linhas a navegadores: **comparar com o estado exibido é sempre mais barato que redesenhar**.

NOTA

5.2 O que importa não é a média

Os 1,3% do Exercício 3.3 podem sugerir que o display é barato. A média engana.

O custo do display não é uma fração de tempo distribuída: é um **bloqueio contínuo** de 1,3 ms. Durante ele, o processador não faz nada além de esperar o controlador — não lê botão, não converte, não alterna pino.

CONCEITO

O número a comparar não é 1,3% do segundo. É 1,3 ms contra a menor coisa que o sistema precisa fazer a tempo.

O curso já tem esse número. O gerador de melodia da bancada produz um lá de 440 Hz invertendo um pino a cada meio período — **1,14 ms**.

Uma atualização de tela é mais longa que meio período do lá. Se as duas coisas convivem no mesmo laço, a nota sai errada toda vez que o display é escrito, e o defeito é audível: um estalo, ou a nota cai de tom.

ATENÇÃO

Nenhuma reorganização do laço resolve. Enquanto a escrita for bloqueante, existe um intervalo de 1,3 ms em que nada mais acontece — e a única saída é o processador ser **interrompido** durante a espera.

É a dívida que o encontro 7 paga.

6 Transposição

Displays modernos são majoritariamente seriais, e a economia de pinos é decisiva. Mudam a camada física e a velocidade; permanecem intactas a separação entre comando e dado e a posição do display como periférico lento do sistema.

Em plataformas com mais memória aparece uma diferença estrutural: o **quadro de memória**. Em vez de escrever caracteres direto no display, o programa desenha numa imagem em RAM e a transfere de uma vez, por acesso direto à memória, sem custo de processador.

NOTA

Isso exige memória que este dispositivo não tem — um display gráfico modesto consome mais RAM do que os 2 kB disponíveis — e é uma das fronteiras concretas entre 8 e 32 bits que os seminários dos encontros 14 e 15 vão revisar.

7 Previsão para o R4

PREVISÃO

P1. Você vai escrever `lcd_texto("Temp: 25.3 C")` e, sem limpar, escrever `lcd_texto("Temp: 9.1 C")`. Desenhe as dezesseis posições da linha 1 como você espera vê-las depois da segunda escrita.

P2. Se você trocar LATD por PORTD na primeira linha de `lcd_nibble`, o display continua funcionando na sua bancada? Diga sim ou não e justifique com o que está ligado ao PORTD do kit.

P3. Você vai remover uma das três repetições de `0x30` da inicialização. Em que condição o display falha, e em que condição ele funciona mesmo assim?

P4. Meça, com o osciloscópio no pino E, a largura do pulso de habilitação. Escreva o valor que espera antes de medir, e diga de onde ele vem.

P5. Você vai trocar o `__delay_ms(2)` da limpeza por `lcd_esperar()` e medir o tempo real com um pino livre levantado antes e abaixado depois. Escreva, antes de medir, se espera um valor maior, menor ou igual a 1,5 ms — e por quê.

P6. Estime o tempo total gasto com o display num sistema que atualiza as duas linhas a cada 100 ms, como percentual do tempo de processador. Repita para 500 ms.

Leve esta folha preenchida e a listagem do driver impressa ou aberta. A previsão é avaliada pelo raciocínio, não por acertar o número.

SEM NOTA

8 Exercícios

Exercício 3.1. Calcule quantos pinos o display consome em oito e em quatro bits, incluindo as linhas de controle, nas duas montagens possíveis: com R/W acionável (a deste kit) e com R/W aterrado. Em seguida verifique, no contrato de pinos do

TAREFA

laboratório, se a versão de oito bits seria viável junto aos demais periféricos do projeto.

Exercício 3.2. Um estudante escreve `lcd_texto("Temp: 25.3 C")` e, em seguida, `lcd_texto("Temp: 9.1 C")`, sem limpar a tela. Descreva exatamente o que aparece no display e proponha duas correções: uma na função de escrita e outra na formatação do número.

TAREFA

Exercício 3.3. Estime o tempo gasto com o display num sistema que atualiza duas linhas completas a cada 100 ms, e expresse como percentual do tempo de processador. Repita para 500 ms e comente a diferença.

TAREFA

Exercício 3.4. Explique por que `lcd_nibble` lê LATD em vez de PORTD, e descreva concretamente o defeito que ocorreria com a leitura da porta caso os quatro pinos baixos estivessem ligados a uma carga capacitiva.

TAREFA

Exercício 3.5. A sequência de inicialização envia três vezes o mesmo valor antes de comutar o modo.

TAREFA

(a) Quantos estados distintos o controlador pode ocupar quando o microcontrolador é reiniciado sem que o display seja desenergizado? Liste-os.

(b) Mostre, para cada estado, o que acontece a cada um dos três pulsos.

(c) Duas repetições bastariam? E quatro seriam mais seguras?

Exercício 3.6. O driver mostrado no início desta aula tinha a espera de 50 μ s dentro de `lcd_pulso`, e não uma espera de 40 μ s por byte.

TAREFA

(a) Quanto custava um byte na versão antiga e quanto custa na corrigida?

(b) Qual era o tempo de uma atualização completa de tela na versão antiga?

(c) A derivação de 1,3 ms da seção 7 valia para qual das duas versões?

Exercício 3.7. Um estudante propõe: “se o display é o gargalo, é só aumentar o clock do processador”. Avalie a proposta. O que melhora, o que não melhora, e por quê?

TAREFA

Exercício 3.8. O driver desta aula espera `__delay_us(40)` depois de cada byte e `__delay_ms(2)` depois da limpeza, mas o kit permite perguntar ao controlador.

TAREFA

(a) Levante um pino livre imediatamente antes do comando de limpeza e abaixe-o imediatamente depois de `lcd_esperar()` retornar. Meça a largura do pulso no osciloscópio.

(b) O valor medido é maior ou menor que os 1,5 ms da folha de dados? Explique a diferença — e diga se ela é uma propriedade do HD44780 ou do módulo específico que está na sua bancada.

(c) Se o valor medido for confortavelmente menor, um colega propõe substituir `lcd_esperar()` por um `__delay_ms` com o valor que você mediu, “que é mais barato em código”. Avalie a proposta.

No encontro 4: aquisição analógica. O número afirmado na tabela desta aula — 60 ciclos por conversão — sai de uma conta com o tempo de aquisição e o tempo de decisão do conversor, e o caminho até ele passa por uma descoberta desconfortável: o pino, que este curso vinha tratando como um bit desde o encontro 0, não é um bit.

NOTA